

NVGPU 的 Async Copy 指令功能与 MLIR 实现

本文主要参考 Nvidia 官方文档。

CUDA 是用于 Nvidia GPU 的类 C 编程模型, CUDA 线程按线程层次结构组织, 存储层次结构中的级别对应于线程层次结构。CUDA 提供三个级别的编程抽象: 线程、线程块和网格。硬件的层次结构与 CUDA 编程模型的映射关系存在三种不同级别的硬件资源: ALU、流多处理器 (SM) 和 GPU。以 Volta V100 为例, 一个 V100 GPU 由 80 个 SM 组成; 一个 SM 被划分为 4 个处理块, 每个处理块由若干 ALU 组成, 例如 16 个 FP32 内核。

线程是最基本的编程抽象和执行单元, 每个线程通常可以访问 255 个寄存器。线程块中的线程数量受架构限制, 同一线程块中的线程运行在同一个 SM 上, 共享同一个资源分区 (如共享内存), 并可以通过共享内存、栅栏同步相互通信。在运行时, 一个线程块只能工作在某个 SM 中, 但 SM 并不是以线程块作为执行单元进行管理, 而是将一个线程块划分为多个 warp, 每个 warp 包括若干 (一般为 32 或 64) 个线程。同一 warp 中的线程以单指令多线程 (Single Instruction Multiple Threads, SIMT) 方式执行, 这些线程在不同数据上同时执行相同的指令。V100 的 SM 内部有 4 个 warp 调度器, 对应于一个 SM 中的 4 个处理块。warp 调度器在每个时钟周期选择一条已经准备好的指令, 将其分派到一组专用的算术指令单元执行。例如, 在 INT32 算术指令单元上执行 INT32 操作, 或在 FP32 算术指令单元上执行 FP32 操作。寄存器文件、数据缓存和共享内存在线程块之间进行分区。因此, warp 切换没有成本损失。多个线程块组合形成一个网格 (grid)。网格对应于设备上的活跃 CUDA 内核程序, 同一网格中的所有线程块的线程数量相同。CUDA 的运行时会将一个线程块调度到一个 SM, 一个网格只调度到一个 GPU。显然, 一个网格可能会占用多个 SM。设备上的所有线程, 不论其属于哪个线程块, 都可以访问全局内存。全局内存的容量最大, 但访问延迟更高、吞吐率更低。CUDA 线程层次结构、存储层次结构和硬件资源层次结构的对应关系如下图所示:

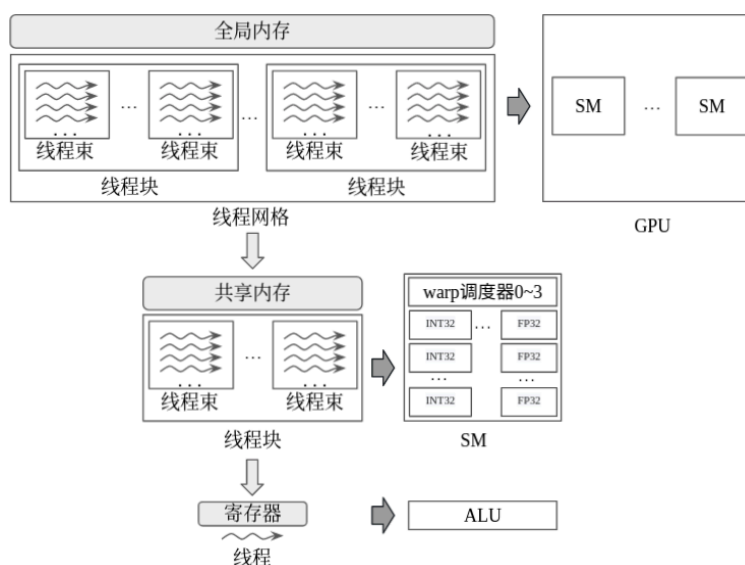


图 CUDA 线程层次结构、存储层次结构和硬件资源层次结构的对应关系

内核 (kernel) 函数通常以一种复制和计算 (copy and compute) 模式执行, 即, 首先从全局内存中获取数据, 并将数据存储到共享内存中, 然后对共享内存数据执行计算, 并将结果 (如果有) 写回全局内存。从 NVIDIA Ampere GPU 架构开始, CUDA 编程模型通过异步编程模型为内存操作提

供加速。异步编程模型定义了与CUDA线程相关的异步操作，包括用于线程间同步的异步栅栏，以及从全局内存中异步搬移数据的异步复制。

异步操作在 CUDA 编程模型中的含义是由某个 CUDA 线程启动的操作，该操作可以像另一个线程一样异步执行。为了保证数据可见性，异步操作使用同步对象来同步操作的完成结果。CUDA 提供了各种不同的同步对象。这些同步对象可以在不同的线程范围（即，使用同步对象与异步操作同步的线程集）内使用。CUDA 中定义的线程范围包括线程、线程块、设备和系统。

如果不使用异步复制指令，则形如“`shared[local_idx] = global[global_idx]`”的语句表示的从全局内存到共享内存的数据复制操作在具体实现时被扩展为先从全局内存读取数据到寄存器，然后将数据从寄存器写入共享内存。为了防止在计算阶段开始之前对共享内存的写入未完成，“`shared[local_idx] = global[global_idx]`”语句之后应进行同步，等待复制操作完成。在计算阶段完成之后还应再次同步，等待计算完成，以防在所有线程完成计算之前其它操作覆盖共享内存。不使用异步复制指令的示例代码如下：

```
for (size_t batch = 0; batch < batch_sz; ++batch) {
    shared[block.thread_rank()] = global_in[batch * block.size() + block.thread_rank()];
    block.sync();
    compute(shared);
    block.sync();
}
```

英伟达在 2020 年 5 月发布的 Ampere GPU 架构中引入了新的异步复制 (Async Copy) 指令，可将全局数据直接从全局内存（通常来自 L2 缓存和 DRAM）加载到 SM 共享内存中。异步复制指令可以在 SM 执行其他计算时在后台完成。在 Volta 架构中，全局数据首先通过全局加载指令由 L1 缓存加载到寄存器文件中，然后通过存储共享指令，将数据从寄存器文件传输到共享内存。最后通过加载共享指令，将数据从共享内存加载到寄存器中。新的异步复制指令通过绕开 L1 缓存，避免数据在寄存器文件中的往返传输，节省了 SM 内部带宽，而且避免了为正在传输的数据分配寄存器文件存储空间。对于大量的、连续的、从全局内存到共享内存的复制操作，使用异步复制可明显提高内存复制性能。

为了实现高效的数据搬移，Ampere 架构还引入了新的共享内存异步栅栏 (Asynchronous Barrier) 指令。该指令是一种内存栅栏 (memory barrier，以下简称为 mbarrier) 指令，可将栅栏的 arrive 和 wait 操作分开，并与异步复制指令协同工作，从而将从全局内存到共享内存的异步复制操作与 SM 中进行计算操作重叠。异步栅栏提供了灵活的、不同粒度的 CUDA 线程同步机制，其同步范围不仅仅局限于 warp 或线程块级别。异步复制操作可以通过新的 mbarrier 通知内核函数数据复制完成。

1. 异步复制的功能与 MLIR 相关实现

采用异步复制的内核函数通常需要软件流水线机制的配合。软件流水线机制将内核函数复制和计算执行模式的主循环分为若干个流水线阶段。上游流水线阶段在加载数据的同时，下游流水线阶段可执行计算，计算使用的操作数来自前一上游流水线阶段的加载操作。在这样的流水线结构中，SM 在使用共享内存中的数据执行当前计算的同时，线程块还应从全局内存中读取下一批次 (batch) 计算所需数据。因此，在计算的共享内存级别采用了双缓冲模式，以保证在上游流水线阶段将数据写入共享内存的同时，下游流水线阶段可以从共享内存加载数据。线程块中的每个线程

复制当前批次数据中的一个或多个元素，然后所有线程同步 (`_syncthreads()` 或 `cooperative_group::sync()` API) 等待所有元素复制操作完成。软件流水线结构下图所示：

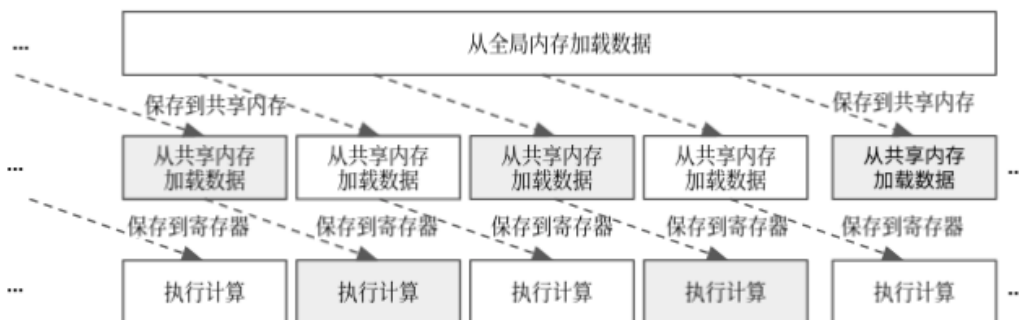


图 软件流水线

在软件流水线结构中，每个线程可以一次或多次调用 `cuda::memcpy_async` 为当前批次数据中的元素提交异步复制操作，然后所有线程等待已经提交的复制操作完成。通过这种方式，可以在多个批次的数据上同时进行搬移和计算。这种处理方式尤其适合 AI 模型中常见的大型数据结构。如果将这类大型数据结构分为 N 个数据批次，线程块可以通过提交 N 个阶段的异步复制，实现对这 N 个批次数据的流水线化的迭代处理，同时也避免了编译器的动态循环展开。使用异步复制指令的示例代码如下：

```

for (size_t batch = 0; batch < batch_sz; ++batch) {
    cooperative_groups::memcpy_async(block, shared,
                                     &global_in1[batch * block.size()], sizeof(T) * block.size());
    cooperative_groups::memcpy_async(block, shared + block.size(),
                                     &global_in2[batch * block.size()], sizeof(T) * block.size());
    cooperative_groups::wait(block);
    compute(shared);
    block.sync();
}

```

此处的 `memcpy_async()` 和 `wait()` 代替了原来的 `memcpy()` 和 `sync()`。

1.1 cp.async 系列指令功能

异步复制指令的行为类似于执行单独的加载全局指令和存储共享指令，但不会消耗寄存器用于数据暂存，这就可以将更多寄存器释放出来供计算使用，并将线程块从复制数据的任务中释放出来。减少寄存器消耗意味着寄存器压力更小，可以改善 GPU 占用率。通过异步复制指令可以一次复制 4、8 和 16 个字节，每个线程可以独立地对全局内存和共享内存寻址。因此，kernel 可以使用异步栅栏指令（异步栅栏指令介绍见第二节），确保正确的写入顺序，以及线程块中各线程间的数据加载和存储的可见性。使用异步栅栏指令同步数据传输的示例代码如下：

```

for (size_t batch = 0; batch < batch_sz; ++batch) {
    cooperative_groups::memcpy_async(block, shared,
                                     &global_in1[batch * block.size()], sizeof(T) * block.size());

```

```

cooperative_groups::memcpy_async(block, shared + block.size(),
                                  &global_in2[batch * block.size()], sizeof(T) * block.size());
barrier.arrive_and_wait();
compute(shared);
barrier.arrive_and_wait();
}

```

复杂的算法需要通过栅栏实现不同级别的同步。通常，开发者可以在 CUDA 中使用线程块同步和设备范围同步两种同步方法。需要注意 `memcpy_async()` 与 `cudaMemcpyAsync()` API 的区别。后者的作用是在启动内核函数之后，利用 CPU 端隐式栅栏实现设备范围同步，并在 CPU 内存和 GPU 全局内存之间异步复制数据，使得 CPU 内存和 GPU 全局内存之间的数据移动可以与 GPU 上的内核执行重叠。

在计算能力 8.0 及以上的设备上，可以使用 `cp.async` 系列指令实现 `memcpy_async()` API。

`cp.async` 指令语法如下：

```

cp.async.ca.shared.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], cp-size{, src-size}{, cache-policy} ;
cp.async.cg.shared.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], 16{, src-size}{, cache-policy} ;
cp.async.ca.shared.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], cp-size{, ignore-src}{, cache-policy} ;
cp.async.cg.shared.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], 16{, ignore-src}{, cache-policy} ;

```

`cp.async` 指令启动内存复制操作，并且在复制操作完成前就将控制返回给执行线程。执行线程可以使用 `cp.async.wait_all`、`cp.async.wait_group` 或 `mbarrier` 指令等待异步复制操作完成，并保证不同异步复制操作之间的顺序。

根据是否访问 L1 缓存，异步复制指令针对不同使用场景可以分为两种模式，如下图所示。图中的灰色方框表示数据复制过程绕过的模块。图 a 是不使用异步复制指令的数据复制过程。在此过程中，全局数据需要在 L1 缓存和寄存器文件中暂存。图 b 是异步复制指令的 BYPASS 模式。在该模式中，全局数据绕过 L1 缓存和寄存器文件，从全局内存经 L2 缓存直接复制到共享内存。图 c 是异步复制指令的 ACCESS 模式。该模式将全局数据保存到 L1 缓存中供后续访问和重用，再复制到共享内存。

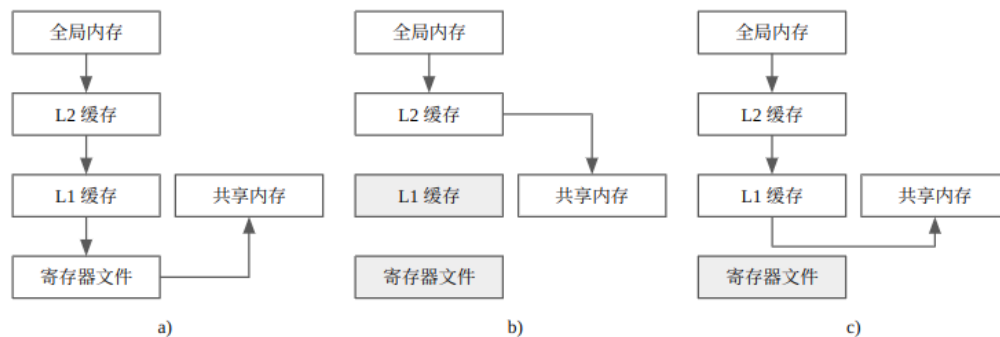


图 异步复制的不同模式

cp.async 指令的限定符 .cg 表示仅在全局级别缓存 L2 而不在 L1 缓存数据, 对应上图中的 BYPASS 模式。限定符 .ca 表示在所有级别缓存数据, 包括 L1 缓存, 对应上图中的 ACCESS 模式。

cp.async 指令的限定符 .level::cache_hint 仅支持 .global 状态空间和地址指向 .global 状态空间的通用寻址, 允许的取值为 .L2::cache_hint。

cp.async 指令的限定符 .level::prefetch_size 指示将指定大小的附加数据提取到相应的缓存级别, 允许的取值为 .L2::64B、.L2::128B、.L2::256B, 表示允许预取的数据大小分别为 L2 中的 64 字节、128 字节或 256 字节。

cp.async 指令的操作数 src 和 dst 分别指定异步复制操作的源地址和目的地址。操作数 src 指定为全局状态空间中的位置, dst 指定为共享状态空间中的位置。

cp.async 指令的操作数 cp-size 为整数常量, 指定了要复制到目的地址的数据长度(以字节为单位)。cp-size 的取值为 4、8 和 16。

cp.async 指令的操作数 src-size 为 32 位整数, 表示要从源地址复制到目的地址的有效数据长度(以字节为单位), 并且, src-size 的值必须小于 cp-size。不足 cp-size 大小的数据用零填充。

cp.async 指令的谓词参数 ignore-src 指定是否应完全忽略来自源地址的数据。如果 ignore-src 为 True, 源数据被忽略, 则零将被复制到目的地址。

cp.async 指令的操作数 cache-policy 指定在内存访问期间可能使用的缓存剔除策略。如果指定了参数 cache-policy, 则必须指定限定符 .level::cache_hint。

在 PTX 中, 有两种方法可以等待异步复制操作完成: 调用 cp.async-groups(QS未支持) 和使用 mbarrier 对象。

如果调用 cp.async-groups 等待异步复制操作完成, 由某个线程的 cp.async 指令启动的一系列异步复制操作被打包提交到该线程拥有的 cp.async-group 中。提交操作由

cp.async.commit_group 指令完成。cp.async.commit_group 指令语法简单, 不再赘述。

cp.async.commit_group 提交操作会为执行线程创建一个新的 cp.async-group, 其中包含由执行线程在创建 cp.async-group 之前发起的所有异步复制操作, 但不包括提交操作之后的任何异步复制操作。以下示例中将两个 cp.async 操作打包进同一个 cp.async-group:

```
cp.async.ca.shared.global [shrd1], [gbl1], 8;  
cp.async.cg.shared.global [shrd1+8], [gbl1+8], 8;  
cp.async.commit_group ;
```

已提交的异步复制操作只属于某一个 cp.async-group。启动异步复制操作的执行线程使用 cp.async.wait_all 或 cp.async.wait_group 等待 cp.async-group 完成后, 才可以访问复制操作写入的数据。cp.async-group 完成意味着属于该 cp.async-group 的所有异步复制操作都已完成。

cp.async-groups 的完成顺序由执行线程提交 cp.async-groups 的顺序决定。

cp.async.wait_group / cp.async.wait_all 指令语法如下:

```
cp.async.wait_group N;  
cp.async.wait_all ;
```

cp.async.wait_group 指令使得执行线程进入等待状态, 直到执行线程之前提交的所有 cp.async-groups 都已完成。cp.async.wait_group 指令的参数 N 指定了最接近

cp.async.wait_group 调用的、可以处于未决状态的 cp.async-groups 数量。如果 N 为 0, 则执行线程应等待所有之前的 cp.async-groups 完成。

以下示例中有三个 cp.async-group。cp.async.wait_group 指令的参数 N 等于 1, 因此只需要等待前两个 cp.async-group 完成, 最后一个 cp.async-group 可以不完成。

```
cp.async.ca.shared.global [shrd0], [gbl0], 4;  
cp.async.cg.shared.global [shrd1], [gbl1], 16;  
cp.async.commit_group; // End of group 0
```

```
cp.async.cg.shared.global [shrd2], [gbl2], 8;  
cp.async.cg.shared.global [shrd3], [gbl3], 16;  
cp.async.commit_group; // End of group 1
```

```
cp.async.cg.shared.global [shrd4], [gbl4], 8;  
cp.async.cg.shared.global [shrd5], [gbl5], 16;  
cp.async.commit_group; // End of group 2
```

```
cp.async.wait_group 1; // waits for group 0 and group 1 to complete
```

各指令的执行顺序示意图如下：

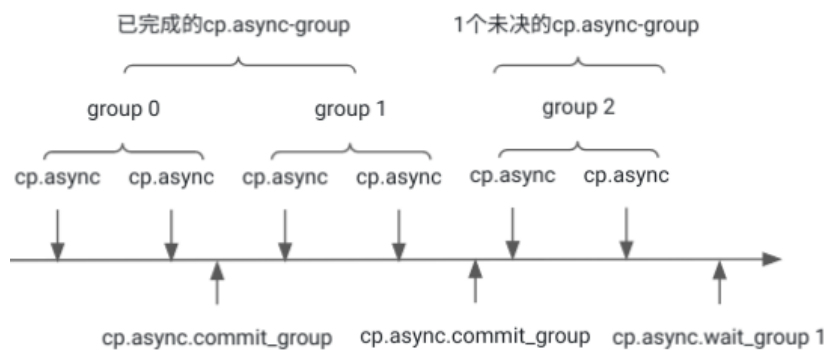


图 异步复制相关指令执行顺序

cp.async.wait_all 相当于 cp.async.commit_group 和 cp.async.wait_group 0 的功能合并。
cp.async.wait_all 的使用示例如下：

```
cp.async.ca.shared.global [shrd0], [gbl0], 4;  
cp.async.cg.shared.global [shrd1], [gbl1], 16;  
cp.async.wait_all;
```

上例中的 cp.async.wait_all 会等待之前启动的所有 cp.async 完成。只有当 cp.async.wait_all 完成或在 cp.async 所属的 cp.async-group 上完成 cp.async.wait_group, cp.async 操作执行的写入数据才对执行线程可见。

如果使用 mbarrier 对象等待异步复制操作完成，异步复制操作在启动后由 mbarrier 对象跟踪异步复制操作。在 mbarrier 对象上等待的线程在调用 mbarrier.test_wait 返回为 True 后，可以访问复制操作写入的数据。

2. 异步复制功能在 MLIR 中的实现

与《MLIR 编译技术内幕》4.2.2 节中介绍的 vector.transfer_read、vector.contract 操作转换为 nvgpu.ldmatrix、nvgpu.mma.sync 操作过程类似，cp.async 系列指令对应的 NVGPU 操作也由 Vector 操作转换得到，并可由前端框架直接通过调用 OpBuilder 的 create() 接口生成。IREE 的 LLVMGPUVectorToGPU pass 在将 Vector 操作转换为 NVGPU 操作的过程中，为了将复制数据到共享内存的操作转换为异步复制操作，调用 createAsyncGroups() 函数(见 <iree_root>/compiler/src/iree/compiler/Codegen/LLVMGPU/Utils/LLVMGPUUtils.cpp)通过 OpBuilder.create() 接口生成异步复制操作及等待操作，从而将全局内存到共享内存的向量数据复制转换为异步复制。

createAsyncGroups() 函数代码实现如下：

```
void createAsyncGroups(RewriterBase &rewriter, mlir::FunctionOpInterface funcOp,
                      bool useMMASync) {
  funcOp.walk([&](Operation *writeOp) {
    if (!isContiguousStore(writeOp))
      return WalkResult::advance();
    Value vectorVal = getValueStored(writeOp);
    if (llvm::cast<VectorType>(vectorVal.getType()).getRank() != 1) {
      return WalkResult::advance();
    }
    Value memrefOperand = getMemrefOperand(writeOp);
    if (!hasSharedMemoryAddressSpace(
        llvm::cast<MemRefType>(memrefOperand.getType()))) {
      return WalkResult::advance();
    }
    Operation *readOp = vectorVal.getDefiningOp();
    if (readOp == nullptr || !isContiguousRead(readOp)) {
      return WalkResult::advance();
    }
    ...
    VectorType vecType = llvm::cast<VectorType>(vectorVal.getType());
    Value storeBase = getMemrefOperand(writeOp);
    Value loadBase = getMemrefOperand(readOp);
    if (!resultsInSupportedAsyncCopy(cast<MemRefType>(loadBase.getType()),
                                     getIndices(readOp), vecType) ||
        !resultsInSupportedAsyncCopy(cast<MemRefType>(storeBase.getType()),
                                     getIndices(writeOp), vecType))
      return WalkResult::advance();

    copyToSharedMem.insert(writeOp);
```

```

    return WalkResult::advance();
});

while (!copyToSharedMem.empty()) {
    SmallVector<Operation *> group;
    Operation *writeOp = *copyToSharedMem.begin();
    copyToSharedMem.remove(writeOp);
    group.push_back(writeOp);
    Operation *nextNode = writeOp;
    while ((nextNode = nextNode->getNextNode())) {
        auto memInterface = dyn_cast<MemoryEffectOpInterface>(nextNode);
        if (memInterface && memInterface.hasNoEffect() &&
            !nextNode->hasTrait<OpTrait::HasRecursiveMemoryEffects>())
            continue;

        if (isa<vector::TransferReadOp, vector::LoadOp>(nextNode)) {
            Operation *readOp = nextNode;
            Value memrefOperand = getMemrefOperand(readOp);
            if (!hasSharedMemoryAddressSpace(
                llvm::cast<MemRefType>(memrefOperand.getType()))) {
                continue;
            }
        }
        if (copyToSharedMem.count(nextNode)) {
            copyToSharedMem.remove(nextNode);
            group.push_back(nextNode);
            continue;
        }
        break;
    }

    SmallVector<Value> tokens;
    for (Operation *writeOp : group) {
        rewriter.setInsertionPoint(writeOp);
        Value vectorVal = getValueStored(writeOp);
        auto vectorType = llvm::cast<VectorType>(vectorVal.getType());
        int64_t numElements = vectorType.getNumElements();
        Operation *readOp = vectorVal.getDefiningOp();
        Value storeBase = getMemrefOperand(writeOp);
        Value loadBase = getMemrefOperand(readOp);
        Value mask = getMaskValue(rewriter, readOp);
        auto dstMemref = llvm::cast<MemRefType>(storeBase.getType());
        int64_t sizeInBytes =
            (dstMemref.getElementTypeBitWidth() * numElements) / 8;
    }
}

```



```

UnitAttr bypassL1 =
    useMMASync && sizeInBytes == 16 ? rewriter.getUnitAttr() : UnitAttr();
Value token = rewriter.create<nvgpu::DeviceAsyncCopyOp>(
    writeOp->getLoc(),
    nvgpu::DeviceAsyncTokenType::get(funcOp.getContext()), storeBase,
    getIndices(writeOp), loadBase, getIndices(readOp),
    rewriter.getIndexAttr(numElements), mask, /*bypassL1=*/bypassL1);
tokens.push_back(token);
}
Value groupToken = rewriter.create<nvgpu::DeviceAsyncCreateGroupOp>(
    funcOp.getLoc(), nvgpu::DeviceAsyncTokenType::get(funcOp.getContext()),
    tokens);
rewriter.create<nvgpu::DeviceAsyncWaitOp>(funcOp.getLoc(), groupToken,
    nullptr);
...
}
}

```

IREE 中实现的 `createAsyncGroups()` 函数与 MLIR 中实现的 `createAsyncGroups()` 函数(见 `<llvm_root>/mlir/lib/Dialect/NVGPU/Transforms/ CreateAsyncGroups.cpp`)函数逻辑基本相同。该函数在指定函数操作(`funcOp`)中查找合适的向量传输操作并将其转换为 `nvgpu.device_async_copy` 操作。如果有连续 `nvgpu.device_async_copy` 操作, 则这些异步复制操作应被放入同一组(`cp.async-group`)中, 并在该组最后插入等待操作。由于只有 16 字节传输可以绕过 L1 缓存, 因此, 如果函数第三个参数 `useMMASync` 或 `bypassL1` (IREE 和 MLIR 的函数定义使用了不同函数参数名)设置为 `true`, 则应根据该参数值为异步复制操作设置合适的传输方式属性, 即使用 16 字节传输。其他传输大小不应此参数值设置为 `true`。

为了充分理解异步复制操作生成过程, 本节使用以下测试用例 `nvgpu_async.mlir` 辅助分析 LLVMGPUVectorToGPU pass 调用 `createAsyncGroups()` 函数的执行步骤。

```

builtin.module {
  func.func @copies_to_asyncs(%a: memref<1024x1024xf32>) {
    %0 = memref.alloc() : memref<4x32x16xf32, #gpu.address_space<workgroup>>
    %c0 = arith.constant 0 : index
    %c4 = arith.constant 4 : index
    %cst_0 = arith.constant 0.000000e+00 : f32
    %1 = vector.transfer_read %a[%c0, %c0], %cst_0 {in_bounds = [true]} :
    memref<1024x1024xf32>, vector<4xf32>
    vector.transfer_write %1, %0[%c0, %c0, %c0] {in_bounds = [true]} : vector<4xf32>,
    memref<4x32x16xf32, #gpu.address_space<workgroup>>
    %2 = vector.transfer_read %a[%c0, %c4], %cst_0 {in_bounds = [true]} :
    memref<1024x1024xf32>, vector<1xf32>
    vector.transfer_write %2, %0[%c0, %c4, %c0] {in_bounds = [true]} : vector<1xf32>,
    memref<4x32x16xf32, #gpu.address_space<workgroup>>
    return
  }
}

```

```

}
}
...

```

createAsyncGroups() 函数功能分为两部分。首先, funcOp.walk() 方法遍历 funcOp 中的每个写操作(如 vector.transfer_write), 并对每个写操作执行 lambda 函数, 找到满足条件的 vector.transfer_write 操作。

由于 createAsyncGroups() 函数的目的是将 funcOp 中符合条件的写操作转换为 nvgpu.device_async_copy 操作, 因此, funcOp.walk() 方法首先应确保写操作是一维数据的连续存储(contiguous store)操作, 且操作的 memref 操作数具有共享内存空间属性。

连续存储是指数据在内存地址空间中连续存放, 中间不出现地址间隔。这种存储方式有助于提高矢量化操作数据访问的效率。连续存储条件检查由 isContiguousStore() 函数实现, 即, 确保操作为次要恒等映射, 没有重排(isMinorIdentity); 存储访问在内存范围内(isDimInBounds); 传输操作是完整的、连续的, 且没有掩码(getMask)。

上述示例代码中的两个 vector.transfer_write 操作的源操作数向量类型分别为 vector<4xf32> 和 vector<1xf32>, 满足一维数据连续存储条件。两个 vector.transfer_write 操作的 memref 操作数的内存空间属性都为 workgroup, 满足共享内存空间条件:

```

vector.transfer_write %1, %0[%c0, %c0, %c0] {in_bounds = [true]} : vector<4xf32>,
memref<4x32x16xf32, #gpu.address_space<workgroup>>
vector.transfer_write %2, %0[%c0, %c4, %c0] {in_bounds = [true]} : vector<1xf32>,
memref<4x32x16xf32, #gpu.address_space<workgroup>>

```

funcOp.walk() 方法接下来检查定义 vector.transfer_write 源操作数的读操作(如 vector.transfer_read)是否为连续读操作(isContiguousRead)。如果不是, 则跳过该操作, 确保内存访问的高效。上述示例代码中的两个 vector.transfer_read 操作都满足连续读操作条件。

上一小节中已经说明, cp.async 指令的操作数 cp-size 取值为 4、8 和 16 字节, 指定了要复制到目的地址的数据长度。funcOp.walk() 方法的第三个检查条件是调用函数 resultsInSupportedAsyncCopy(), 确定 nvgpu.device_async_copy 操作支持 vector.transfer_write 操作的源操作数类型。上述示例代码中的两个 vector.transfer_write 操作的源操作数向量类型分别为 vector<4xf32> 和 vector<1xf32>, 对应 cp-size 取值分别为 16 和 4 字节, nvgpu.device_async_copy 操作可以支持。

至此, 示例代码中的两个 vector.transfer_write 操作均满足 funcOp.walk() 方法规定的三个检查条件:

第一, vector.transfer_write 操作是一维数据的连续存储操作且 memref 操作数具有共享内存空间属性;

第二, 对应的 vector.transfer_read 操作是连续读操作;

第三, vector.transfer_write 操作的源操作数向量类型满足 cp-size 取值范围。

满足条件的两个 vector.transfer_write 操作被保存在操作集合 copyToSharedMem 中。

在找到满足条件的 vector.transfer_write 操作并将其保存在集合 copyToSharedMem 中以后, createAsyncGroups() 函数的第二部分功能是遍历 copyToSharedMem 集合中的每个 vector.transfer_write 操作, 将其中连续的共享内存写 vector.transfer_write 操作放在同一个组(对应 cp.async-group)中, 即在 vector.transfer_write 操作后插入

nvgpu.device_async_create_group 操作，并在 cp.async-group 的最后插入 nvgpu.device_async_wait 操作。

判断若干个 vector.transfer_write 操作为连续共享内存写操作的方法是通过调用 getNextNode() 接口，遍历 copyToSharedMem 集合中第一个 vector.transfer_write 操作之后的操作，并将有副作用的操作 (hasNoEffect) 和非共享内存地址空间的读操作过滤掉。

例如，经过 createAsyncGroups() 函数第一部分功能的条件筛选后，copyToSharedMem 集合中的第一个 vector.transfer_write 操作为：

```
vector.transfer_write %1, %0[%c0, %c0, %c0] {in_bounds = [true]} : vector<4xf32>,
memref<4x32x16xf32, #gpu.address_space<workgroup>>
```

调用 getNextNode() 接口得到的下一个操作是如下 vector.transfer_read 操作：

```
%2 = vector.transfer_read %a[%c0, %c4, %c0] {in_bounds = [true]} :
memref<1024x1024xf32>, vector<1xf32>
```

该 vector.transfer_read 操作的 memref 操作数不具有共享内存空间属性，因此不需要进一步处理，继续遍历下一个 vector.transfer_write 操作：

```
vector.transfer_write %2, %0[%c0, %c4, %c0] {in_bounds = [true]} : vector<1xf32>,
memref<4x32x16xf32, #gpu.address_space<workgroup>>
```

该 vector.transfer_write 操作不满足过滤条件，可视为与前一 vector.transfer_write 操作构成连续共享内存写操作。因此，可将这两个 vector.transfer_write 操作打包进同一个组 (cp.async-group) 中，并将组中每一个 vector.transfer_write 操作替换为新构造的 nvgpu.device_async_copy 操作。

为了支持上述 createAsyncGroups() 函数生成 nvgpu.device_async_copy 操作，NVGPU 方言中定义了如下 nvgpu.device_async_copy 操作：

```
def NVGPU_DeviceAsyncCopyOp : NVGPU_Op<"device_async_copy", [
    AttrSizedOperandSegments]> {
    let summary = "device-side asynchronous copy";
    let description = [{ ... }];
    let results = (outs NVGPU_DeviceAsyncToken:$asyncToken);
    let arguments = (ins Arg<AnyMemRef, "", [MemWriteAt<0, FullEffect>]>:$dst,
        Variadic<Index>:$dstIndices,
        Arg<AnyMemRef, "", [MemReadAt<0, FullEffect>]>:$src,
        Variadic<Index>:$srcIndices,
        IndexAttr:$dstElements,
        Optional<Index>:$srcElements,
        OptionalAttr<UnitAttr>:$bypassL1);
    let assemblyFormat = [{ ... }];
    let hasVerifier = 1;
}
```

其中, 操作结果 `asyncToken` 的类型为 `NVGPU_DeviceAsyncToken`, 可以用于在异步操作 (如 `nvgpu.device_async_copy` 操作) 和对异步操作进行分组或同步的操作 (如 `nvgpu.device_async_create_group`、`nvgpu.device_async_wait`) 之间建立基于 SSA 的链接。操作参数 `dst` 为异步复制的目的 `memref`; 操作参数 `dstIndices` 为目的 `memref` 索引; 操作参数 `src` 为异步复制的源 `memref`; 操作参数 `srcIndices` 为源 `memref` 索引; 操作参数 `dstElements` 为复制到目的地址的元素数量; 操作参数 `srcElements` 是可选参数, 指定从源拷贝的元素数量。如果未提供该参数, 默认使用 `dstElements` 值。操作参数 `bypassL1` 是可选参数, 如果指定该值, 表示在异步复制操作绕过 L1 缓存。

下图举例说明了 `createAsyncGroups()` 函数在生成 `nvgpu.device_async_copy` 操作时该操作各字段 (图中的灰色矩形框) 的获取来源:

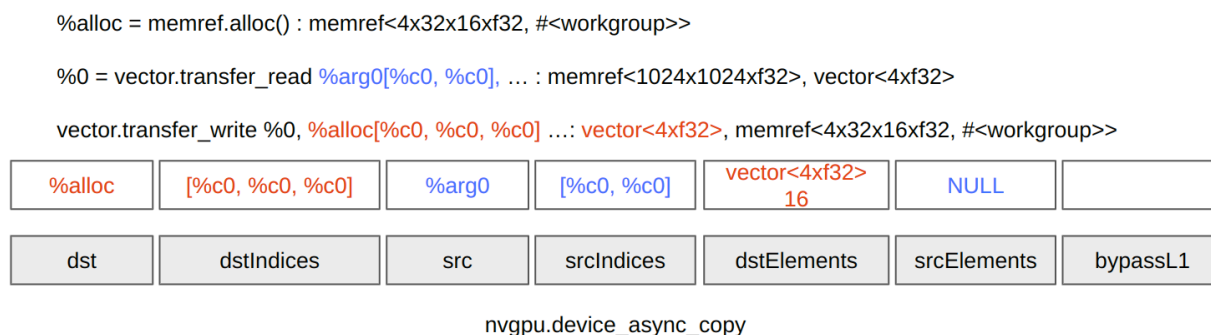


图 `nvgpu.device_async_copy` 操作各字段的来源

从图中可以看到, `nvgpu.device_async_copy` 操作中与目的相关的各字段来自 `vector.transfer_write` 操作 (图中的红色内容), 与源相关的各字段来自 `vector.transfer_read` 操作的操作数 (图中的蓝色内容)。

将所有 `vector.transfer_write` 操作替换为 `nvgpu.device_async_copy` 操作后, 还需要再构造 `nvgpu.device_async_create_group` 操作, 创建新的 `cp.async-group`, 并构造 `nvgpu.device_async_wait` 操作, 使得执行线程进入等待状态。

上述示例代码经过 `iree-opt` 工具命令行:

```
tools/iree-opt nvgpu_async.mlir -iree-transform-dialect-interpreter -split-input-file
--verify-diagnostics
```

处理后输出代码如下:

```
module {
  module {
    func.func @copies_to_asyncs(%arg0: memref<1024x1024xf32>) {
      %alloc = memref.alloc() : memref<4x32x16xf32, #gpu.address_space<workgroup>>
      %c0 = arith.constant 0 : index
      %c4 = arith.constant 4 : index
      %cst = arith.constant 0.000000e+00 : f32
```

```

    %0 = vector.transfer_read %arg0[%c0, %c0], %cst {in_bounds = [true]} :
memref<1024x1024xf32>, vector<4xf32>
    %1 = nvgpu.device_async_copy %arg0[%c0, %c0], %alloc[%c0, %c0, %c0], 4 {bypassL1}
: memref<1024x1024xf32> to memref<4x32x16xf32, #gpu.address_space<workgroup>>
    %2 = vector.transfer_read %arg0[%c0, %c4], %cst {in_bounds = [true]} :
memref<1024x1024xf32>, vector<1xf32>
    %3 = nvgpu.device_async_copy %arg0[%c0, %c4], %alloc[%c0, %c4, %c0], 1 :
memref<1024x1024xf32> to memref<4x32x16xf32, #gpu.address_space<workgroup>>
    %4 = nvgpu.device_async_create_group %1, %3
    nvgpu.device_async_wait %4
    return
}
}
...
}

```

上述输出代码中的两个 `nvgpu.device_async_copy` 操作由对应的两个 `vector.transfer_write` 操作替换得到。由于这两个 `vector.transfer_write` 操作构成连续共享内存写操作，`createAsyncGroups()` 函数生成 `nvgpu.device_async_create_group` 操作和 `nvgpu.device_async_wait` 操作。

3. `nvgpu.device_async_copy` 操作到 `nvvm.cp.async.shared.global` 操作的转换
`ConvertNVGPToNVVMPass` 的 `runOnOperation()` 函数中定义了 `RewritePatternSet` 实例 `patterns`，其中包含了 NVGPU 操作递降为 NVVM 操作时用到的各种模式。`runOnOperation()` 函数调用 `populateNVGPToNVVMConversionPatterns()` 函数，向重写模式集合中添加各种模式，可将 `nvgpu.device_async_copy` 操作转换为 `nvvm.cp.async.shared.global`（以下简称为 `nvvm.cp.async`）操作的重写模式 `NVGPUAsyncCopyLowering` 也在其中。《MLIR 编译技术内幕》4.3.1 节已经详细介绍 `ConvertNVGPToNVVMPass` 的定义与实现（代码见 `<llvm_root>/mlir/lib/Conversion/NVGPToNVVM/NVGPToNVVM.cpp`）。

:

```

struct ConvertNVGPToNVVMPass
: public impl::ConvertNVGPToNVVMPassBase<ConvertNVGPToNVVMPass> {
...
void runOnOperation() override {
...
    populateNVGPToNVVMConversionPatterns(converter, patterns);
...
}
};

void mlir::populateNVGPToNVVMConversionPatterns(LLVMTypeConverter &converter,
RewritePatternSet &patterns) {
    patterns.add<
..., NVGPUAsyncCopyLowering, ...>(converter);

```

```
}
```

本节主要关注 NVGPUAsyncCopyLowering 重写模式的实现。为了充分理解 nvgpu.device_async_copy 操作到 nvvm.cp.async 操作的递降过程, 本节使用以下测试用例 nvvm_async.mlir 辅助分析 NVGPUAsyncCopyLowering 重写模式的 matchAndRewrite() 函数的执行步骤。

```
func.func @nvvm_async(%src: memref<128x128xf32>, %dst: memref<3x16x128xf32, 3>,
                     %i : index) {
    %0 = nvgpu.device_async_copy %src[%i, %i], %dst[%i, %i, %i], 4 : memref<128x128xf32> to
memref<3x16x128xf32, 3>
    %1 = nvgpu.device_async_create_group %0
    nvgpu.device_async_wait %1 { numGroups = 1 : i32 }
    %2 = nvgpu.device_async_copy %src[%i, %i], %dst[%i, %i, %i], 4 {bypassL1}:
memref<128x128xf32> to memref<3x16x128xf32, 3>
    return
}
```

针对测试用例 nvvm_async.mlir, NVGPUAsyncCopyLowering::matchAndRewrite() 函数(代码实现见<llvm_root>/mlir/lib/Conversion/NVGPToNVVM/NVGPToNVVM.cpp)的代码实现和工作过程分析如下:

struct NVGPUAsyncCopyLowering

```
: public ConvertOpToLLVMPattern<nvgpu::DeviceAsyncCopyOp> {
...
LogicalResult
matchAndRewrite(nvgpu::DeviceAsyncCopyOp op, OpAdaptor adaptor,
                 ConversionPatternRewriter &rewriter) const override {
    ImplicitLocOpBuilder b(op.getLoc(), rewriter);
    Location loc = op.getLoc();
    auto dstMemrefType = cast<MemRefType>(op.getDst().getType());
    Value dstPtr = getStridedElementPtr(b.getLoc(), dstMemrefType, adaptor.getDst(),
                                         adaptor.getDstIndices(), rewriter);
    ...
    auto srcMemrefType = cast<MemRefType>(op.getSrc().getType());
    ...
    Value scrPtr = getStridedElementPtr(loc, srcMemrefType, adaptor.getSrc(),
                                         adaptor.getSrcIndices(), rewriter);
    // Intrinsics takes a global pointer so we need an address space cast.
    auto srcPointerGlobalType = LLVM::LLVMPointerType::get(
        op->getContext(), NVVM::NVVMMemorySpace::kGlobalMemorySpace);
    scrPtr = b.create<LLVM::AddrSpaceCastOp>(srcPointerGlobalType, scrPtr);
    int64_t dstElements = adaptor.getDstElements().getZExtValue();
    int64_t sizeInBytes =
```

```

    (dstMemrefType.getElementTypeBitWidth() * dstElements) / 8;
// When the optional SrcElements argument is *not* present, the regular
// CpAsyncOp is generated. CopyAsyncOp reads bytes from source (global
// memory) to fill DstElements number of elements in the destination
// (shared memory).
Value srcBytes = adaptor.getSrcElements();
if (srcBytes) {
    // When the optional SrcElements argument is present, the source (global
    // memory) of CpAsyncOp is read only for SrcElements number of elements.
    // The rest of the DstElements in the destination (shared memory) are
    // filled with zeros.
    Value c3I32 =
        b.create<LLVM::ConstantOp>(b.getI32Type(), b.getI32IntegerAttr(3));
    Value bitwidth = b.create<LLVM::ConstantOp>(
        b.getI32Type(),
        b.getI32IntegerAttr(srcMemrefType.getElementTypeBitWidth()));
    Value srcElementsI32 = b.create<LLVM::TruncOp>(b.getI32Type(), srcBytes);
    srcBytes = b.create<LLVM::LShrOp>(
        b.create<LLVM::MulOp>(bitwidth, srcElementsI32), c3I32);
}
// Cache global (.cg) for 16 dst bytes, Cache all (.ca) for sizes other than
// 16 dst bytes.
NVVM::LoadCacheModifierKind cacheModifier =
    (op.getBypassL1().value_or(false) && sizeInBytes == 16)
        ? NVVM::LoadCacheModifierKind::CG
        : NVVM::LoadCacheModifierKind::CA;

b.create<NVVM::CpAsyncOp>(
    dstPtr, srcPtr, rewriter.getI32IntegerAttr(sizeInBytes),
    NVVM::LoadCacheModifierKindAttr::get(op->getContext(), cacheModifier),
    srcBytes);

// Drop the result token.
Value zero = b.create<LLVM::ConstantOp>(
    IntegerType::get(op.getContext(), 32), rewriter.getI32IntegerAttr(0));
rewriter.replaceOp(op, zero);
return success();
}
};

```

nvgpu.device_async_copy 操作到 nvvm.cp.async 操作的递降过程同样遵循《MLIR 编译技术内幕》4.3.1 节中提出的操作转换四步骤，即，操作生成前的输入数操作计算与类型转换、生成操作、生成操作后的结果操作数计算与类型转换、操作替换。但因为 nvvm.cp.async 操作没有结果操作数，因而不需要实现操作转换中的结果操作数计算与类型转换。

3.1 nvvm.cp.async 输入操作数的计算和类型转换

NVGPUAsyncCopyLowering::matchAndRewrite() 函数功能的第一部分是为生成 nvvm.cp.async 操作准备必要的输入操作数。nvvm.cp.async 操作定义如下：

```
def NVVM_CpAsyncOp : NVVM_PTXBuilder_Op<"cp.async.shared.global">,
  Arguments<(ins LLVM_PointerShared:$dst,
    LLVM_PointerGlobal:$src,
    I32Attr:$size,
    LoadCacheModifierAttr:$modifier,
    Optional<LLVM_Type>:$cpSize)> {
string llvmBuilder = [{
  llvm::Intrinsic::ID id;
  switch ($size) {
    case 4:
      id = llvm::Intrinsic::nvvm_cp_async_ca_shared_global_4;
      break;
    case 8:
      id = llvm::Intrinsic::nvvm_cp_async_ca_shared_global_8;
      break;
    case 16:
      if($modifier == NVVM::LoadCacheModifierKind::CG)
        id = llvm::Intrinsic::nvvm_cp_async_cg_shared_global_16;
      else if($modifier == NVVM::LoadCacheModifierKind::CA)
        id = llvm::Intrinsic::nvvm_cp_async_ca_shared_global_16;
      else
        llvm_unreachable("unsupported cache modifier");
      break;
    default:
      llvm_unreachable("unsupported async copy size");
  }
  createIntrinsicCall(builder, id, {$dst, $src});
}];
let assemblyFormat = "$dst`, ` $src`, ` $size`, ` `cache` `=` $modifier (` ` $cpSize^)? attr-dict `:`
type(operands)";
let hasVerifier = 1;
let extraClassDeclaration = [{
  bool hasIntrinsic() { if(getCpSize()) return false; return true; }

void getAsmValues(RewriterBase &rewriter,
  llvm::SmallVectorImpl<std::pair<mlir::Value, mlir::NVVM::PTXRegisterMod>> &asmValues)
{
  asmValues.push_back({getDst(), PTXRegisterMod::Read});
  asmValues.push_back({getSrc(), PTXRegisterMod::Read});
  asmValues.push_back({makeConstantI32(rewriter, getSize()), PTXRegisterMod::Read});
```

```

    asmValues.push_back({getCpSize(), PTXRegisterMod::Read});
  }
  });
let extraClassDefinition = [{
  std::string $cppClass::getPtx() {
    if(getModifier() == NVVM::LoadCacheModifierKind::CG)
      return std::string("cp.async.cg.shared.global [%0], [%1], %2, %3;\n");
    if(getModifier() == NVVM::LoadCacheModifierKind::CA)
      return std::string("cp.async.ca.shared.global [%0], [%1], %2, %3;\n");
    llvm_unreachable("unsupported cache modifier");
  }
  });
}

```

由 nvvm.cp.async 操作定义可知，在生成 nvvm.cp.async 操作前需要提供指向共享内存的目的地址指针(dst)、指向全局内存的源地址指针(src)、复制到目的地址的数据长度(size)、缓存修饰符(modifier)和源数据长度(cpSize)五个操作数。其中，nvvm.cp.async 操作的参数 size 根据 nvgpu.device_async_copy 操作的参数 dstElements(对应 cp.async 指令的 cp-size 字段，但单位不同)计算得到，nvvm.cp.async 操作的参数 cpSize 根据 nvgpu.device_async_copy 操作的参数 srcElements(对应 cp.async 指令的 src-size 字段，但单位不同)计算得到。如果 nvgpu.device_async_copy 操作中未提供参数 srcElements，则从源地址复制到目的地址的有效数据长度由参数 dstElements 决定；如果 nvgpu.device_async_copy 操作中提供了参数 srcElements，则从源地址复制到目的地址的有效数据长度由参数 srcElements 决定，且 srcElements 值不能超过 dstElements 值。不足 dstElements 大小的数据用零填充。上述示例 nvvm_async.mlir 经过 mlir-opt 工具命令行：

```
bin/mlir-opt nvvm_async.mlir -convert-nvgpu-to-nvvm
```

处理后输出代码如下：

```

module {
  func.func @nvvm_async(%arg0: memref<128x128xf32>, %arg1: memref<3x16x128xf32, 3>,
    %arg2: index) {
    %0 = builtin.unrealized_conversion_cast %arg1 : memref<3x16x128xf32, 3> to
    !llvm.struct<(ptr<3>, ptr<3>, i64, array<3 x i64>, array<3 x i64>)>
    %1 = builtin.unrealized_conversion_cast %arg2 : index to i64
    %2 = builtin.unrealized_conversion_cast %arg0 : memref<128x128xf32> to !llvm.struct<(ptr,
    ptr, i64, array<2 x i64>, array<2 x i64>)>
    %3 = llvm.extractvalue %0[1] : !llvm.struct<(ptr<3>, ptr<3>, i64, array<3 x i64>, array<3 x
    i64>)>
    %4 = llvm.mlir.constant(2048 : index) : i64
    %5 = llvm.mul %1, %4 : i64
    %6 = llvm.mlir.constant(128 : index) : i64
    %7 = llvm.mul %1, %6 : i64
  }
}

```

```

%8 = llvm.add %5, %7 : i64
%9 = llvm.add %8, %1 : i64
%10 = llvm.getelementptr %3[%9] : (!llvm.ptr<3>, i64) -> !llvm.ptr<3>, f32
%11 = llvm.extractvalue %2[1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
%12 = llvm.mlir.constant(128 : index) : i64
%13 = llvm.mul %1, %12 : i64
%14 = llvm.add %13, %1 : i64
%15 = llvm.getelementptr %11[%14] : (!llvm.ptr, i64) -> !llvm.ptr, f32
%16 = llvm.addrspacecast %15 : !llvm.ptr to !llvm.ptr<1>
nvvm.cp.async.shared.global %10, %16, 16, cache = ca : !llvm.ptr<3>, !llvm.ptr<1>
%17 = llvm.mlir.constant(0 : i32) : i32
nvvm.cp.async.commit.group
%18 = llvm.mlir.constant(0 : i32) : i32
nvvm.cp.async.wait.group 1
%19 = llvm.extractvalue %0[1] : !llvm.struct<(ptr<3>, ptr<3>, i64, array<3 x i64>, array<3 x
i64>)>
%20 = llvm.mlir.constant(2048 : index) : i64
%21 = llvm.mul %1, %20 : i64
%22 = llvm.mlir.constant(128 : index) : i64
%23 = llvm.mul %1, %22 : i64
%24 = llvm.add %21, %23 : i64
%25 = llvm.add %24, %1 : i64
%26 = llvm.getelementptr %19[%25] : (!llvm.ptr<3>, i64) -> !llvm.ptr<3>, f32
%27 = llvm.extractvalue %2[1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
%28 = llvm.mlir.constant(128 : index) : i64
%29 = llvm.mul %1, %28 : i64
%30 = llvm.add %29, %1 : i64
%31 = llvm.getelementptr %27[%30] : (!llvm.ptr, i64) -> !llvm.ptr, f32
%32 = llvm.addrspacecast %31 : !llvm.ptr to !llvm.ptr<1>
nvvm.cp.async.shared.global %26, %32, 16, cache = cg : !llvm.ptr<3>, !llvm.ptr<1>
%33 = llvm.mlir.constant(0 : i32) : i32
return
}
}

```

本节描述的 `nvvm.cp.async` 输入操作数类型转换过程与 4.3.2 节中描述的 `nvvm.ldmatrix` 输入操作数类型转换过程相似。二者不同之处在于，`nvvm.cp.async` 操作涉及更多参数，因此转换过程更加复杂。

`nvvm.cp.async` 输入操作数类型转换的重点是计算该操作的地址指针和源地址指针。`matchAndRewrite()` 函数调用 `getStridedElementPtr()` 接口，生成 `llvm.mul`、`llvm.add` 和 `llvm.getelementptr` 等 LLVM 方言操作，并根据 `nvgpu.device_async_copy` 操作参数 `dstIndices`、`srcIndices` 给定的地址索引和源地址索引，以及由目的操作数和源操作数类型 `dstMemrefType`、`srcMemrefType` 给定的跨步和偏移量，可计算得到 `nvvm.cp.async` 操作的目的

地址指针 dstPtr 和源地址指针 srcPtr。下图描述了 dstPtr 和 srcPtr 在操作数中的位置及其计算过程。

```
%0 = nvgpu.device_async_copy %src[%i, %i], %dst[%i, %i, %i], 4 :  
      memref<128x128xf32> to memref<3x16x128xf32, 3>
```

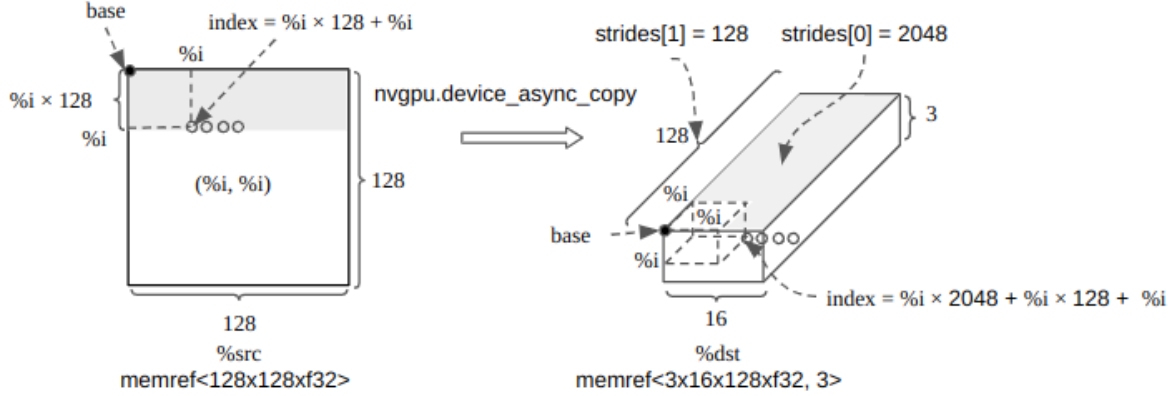


图 nvgpu.device_async 操作的源地址指针和目的地址指针索引计算方法

3.1.1 计算目的地址指针操作数

matchAndRewrite() 函数首先计算 nvgpu.device_async 操作的目的地址指针 dstPtr。与计算 dstPtr 对应的 LLVM 方言操作序列如下：

```
%0 = builtin.unrealized_conversion_cast %arg1 : memref<3x16x128xf32, 3> to  
      !llvm.struct<(ptr<3>, ptr<3>, i64, array<3 x i64>, array<3 x i64>)>  
%1 = builtin.unrealized_conversion_cast %arg2 : index to i64  
...  
%3 = llvm.extractvalue %0[1] : !llvm.struct<(ptr<3>, ptr<3>, i64, array<3 x i64>, array<3 x i64>)>  
%4 = llvm.mlir.constant(2048 : index) : i64  
%5 = llvm.mul %1, %4 : i64  
%6 = llvm.mlir.constant(128 : index) : i64  
%7 = llvm.mul %1, %6 : i64  
%8 = llvm.add %5, %7 : i64  
%9 = llvm.add %8, %1 : i64  
%10 = llvm.getelementptr %3[%9] : (!llvm.ptr<3>, i64) -> !llvm.ptr<3>, f32
```

其中，%0 是 %dst 的 memref 描述符 (MemRefDescriptor)，由类型为 memref<3x16x128xf32, 3> 的 %arg1 经过规范化和转换后得到。该描述符类型为 LLVM 结构 !llvm.struct<(ptr<3>, ptr<3>, i64, array<3 x i64>, array<3 x i64>)>，共有五个字段。此处仅用到第二个字段，即指向数据有效负载的指针。

上述操作序列中的 llvm.extractvalue 操作由 getStridedElementPtr() 函数中调用 MemRefDescriptor 类的 bufferPtr() 接口生成，目的是从 memref 描述符参数 %0 中提取指向共享内存的基地址指针 (即图中 %dst 的 base)。然后，根据 nvgpu.device_async_copy 操作参数

dstIndices 的维数, 多次调用 llvm.mul 和 llvm.add 操作, 计算由 nvgpu.device_async_copy 操作参数 dstIndices 指定的 dstPtr 指针索引。

上述操作序列中的第一个 llvm.mul 操作将 %1(即 %i)与 %4 代表的跨步值 2048 (即 %dst 的横截面元素数量)相乘, 得到 %5(= %i × 2048)。第二个 llvm.mul 操作将 %1(即 %i)与 %6 代表的跨步值 128 (即 %dst 的侧边元素数量)相乘, 得到 %7(= %i × 128)。随后将 %5 与 %7 相加, 再加上 %1(即 %i)得到 dstPtr 指针索引 %9(即上图中 %dst 的 index)。

计算 dstPtr 指针索引后, getStridedElementPtr() 函数调用 rewriter.create<LLVM::GEPop>() 接口生成 llvm.getelementptr 操作。上述操作序列中的 llvm.getelementptr 操作的基地址指针操作数 %3 指向 %dst 在共享内存中的地址, 可使用 %9 作为偏移量计算目的地址指针 dstPtr。

对比 nvvm.cp.async 操作定义和 nvgpu.device_async_copy 操作定义可以发现, nvvm.cp.async 操作定义中没有源和目的 memref 索引参数, 原因是这些索引参数已经计入 nvvm.cp.async 操作的源和目的地址指针参数。

3.1.2 计算源地址指针操作数

matchAndRewrite() 函数接下来计算 nvvm.cp.async 操作的源地址指针 srcPtr。与计算 srcPtr 对应的 LLVM 方言操作序列如下:

```
%11 = llvm.extractvalue %2[1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>>
%12 = llvm.mlir.constant(128 : index) : i64
%13 = llvm.mul %1, %12 : i64
%14 = llvm.add %13, %1 : i64
%15 = llvm.getelementptr %11[%14] : (!llvm.ptr, i64) -> !llvm.ptr, f32
%16 = llvm.addrspacecast %15 : !llvm.ptr to !llvm.ptr<1>
```

srcPtr 的计算过程与 dstPtr 类似, 简要分析如下。%2 是 %src 的 memref 描述符。上述操作序列中的 llvm.extractvalue 操作从 %0 中提取指向全局内存的基地址指针(即上图中 %src 的 base)。然后, 根据 nvgpu.device_async_copy 操作参数 srcIndices 的维数, 多次调用 llvm.mul 和 llvm.add 操作, 计算由 nvgpu.device_async_copy 操作参数 dstIndices 指定的 dstPtr 指针索引。上述操作序列中的 llvm.mul 操作将 %1(即 %i)与 %12 代表的跨步值 128 (即 %src 的行元素数量)相乘, 得到 %13(= %i × 128)。随后将 %13 与 %1 相加, 得到 srcPtr 指针索引 %14(即上图中 %src 的 index)。图中的灰色矩形面积表示指针索引值的大小。

上述操作序列中的 llvm.getelementptr 操作根据基地址指针操作数 %11 和指针索引 %9 计算得到源地址指针 srcPtr, 并由 llvm.addrspacecast 操作将其从通用地址空间类型(llvm.ptr<0>)转换为全局地址空间类型(llvm.ptr<1>)。

对比 nvvm.cp.async 操作定义和 nvgpu.device_async_copy 操作定义可以发现, nvvm.cp.async 操作定义中没有源和目的 memref 索引参数, 原因是这些索引参数已经计入 nvvm.cp.async 操作的源和目的地址指针参数。

3.1.3 nvvm.cp.async 输入其他操作数的计算和类型转换

在生成 nvvm.cp.async 操作前还必须为其计算参数 size、cpSize 和 modifier。

nvvm.cp.async 操作的参数 size 根据 nvgpu.device_async_copy 操作的参数 dstElements 计算得到, 但参数 size 的单位为字节, 参数 dstElements 的单位为元素。因此, 需要根据目的 memref 数据类型和 dstElements 参数值计算得到以字节为单位的复制数据长度。例如, 在 nvvm_async.mlir 测试用例中, nvgpu.device_async_copy 操作的 dstElements 参数值为 4, 目的 memref 数据类型(dstMemrefType)为 memref<3x16x128xf32, 3>:

```
%0 = nvgpu.device_async_copy %src[%i, %i], %dst[%i, %i, %i], 4 : memref<128x128xf32> to
memref<3x16x128xf32, 3>
```

因此，可以计算得到 nvvm.cp.async 操作参数 size (sizeInBytes) 值为 $16 (= 32 \times 4 / 8)$ 。
 nvvm.cp.async 操作的参数 cpSize 根据 nvgpu.device_async_copy 操作的参数 srcElements (对应 cp.async 指令的 src-size 字段，但单位不同) 计算得到。参数 srcElements 是可选参数。如果 nvgpu.device_async_copy 操作没有提供 srcElements 参数，则 nvvm.cp.async 操作从全局内存的源地址读取并复制到共享内存目的地址的有效数据长度由 nvgpu.device_async_copy 操作参数 dstElements 决定；如果 nvgpu.device_async_copy 操作提供了 srcElements 参数，则 nvvm.cp.async 操作从全局内存的源地址读取并复制到共享内存目的地址的有效数据长度由 srcElements 参数决定。不足 dstElements 的部分填充 0。

与上述 vgpu.device_async_copy 操作的 dstElements 参数到 nvvm.cp.async 操作的 size 参数转换类似，srcElements 参数到 cpSize 参数的转换也涉及长度单位由元素转换为字节。本测试用例中的 nvgpu.device_async_copy 操作没有提供 srcElements 参数，因此不需要执行相关转换。需要为 nvvm.cp.async 操作计算的最后一个参数是缓存修饰符 modifier。该参数的取值取决于 nvgpu.device_async_copy 操作是否提供 bypassL1 参数，以及 nvvm.cp.async 操作的 size 参数值。如果 nvgpu.device_async_copy 操作提供了 bypassL1 参数 (表示复制数据不经过 L1 缓存) 且 size 参数值为 16，如下示例：

```
%2 = nvgpu.device_async_copy %src[%i, %i], %dst[%i, %i, %i], 4 {bypassL1}:
memref<128x128xf32> to memref<3x16x128xf32, 3>
```

则 nvvm.cp.async 操作的缓存修饰符参数 modifier(cacheModifier) 取值为枚举属性值 NVVM::LoadCacheModifierKind::CG，表示仅在全局级别缓存 L2 而不在 L1 缓存数据，对应的 cp.async 指令的 BYPASS 模式；如果 nvgpu.device_async_copy 操作没有提供 bypassL1 参数，则参数 modifier 取值为枚举属性值 NVVM::LoadCacheModifierKind::CA，表示在所有级别缓存数据，包括 L1 缓存，对应的 cp.async 指令的 ACCESS 模式。

此外，如果 nvgpu.device_async_copy 操作提供了 bypassL1 参数，size 参数值只能为 16。其他 size 参数值将导致 vgpu.device_async_copy 操作的验证函数报错。

下表总结了当 bypassL1 参数、size 参数取不同值时得到的 modifier 参数值及其含义。

表 bypassL1 参数、size 参数与 modifier 参数值的关系

bypassL1 参数	size 参数(字节)	modifier 取值	含义
无	16	CA	在所有级别缓存数据，包括 L1 缓存。
无	4/8	CA	
有	16	CG	仅在全局级别缓存 L2 而不在 L1 缓存数据。
有	4/8	/	bypassL1 参数与 size 参数冲突。

3.2 生成 nvvm.cp.async 操作

在获得 `nvvm.cp.async` 操作的所有参数 `Ptr` 后, 可调用 `b.create<NVVM::CpAsyncOp>( )` 接口在生成 `NVVM::CpAsyncOp` 类型的操作。最终生成的 `nvvm.cp.async` 操作如下:

```
nvvm.cp.async.shared.global %10, %16, 16, cache = ca : !llvm.ptr<3>, !llvm.ptr<1>
```

由于 `nvvm.cp.async` 操作没有结果操作数, 因此不需要做结果操作数的计算和类型转换。
测试用例中的其他 NVGPU 异步操作, 如 `nvgpu.device_async_create_group` 操作和 `nvgpu.device_async_wait` 操作的实现和递降过程较为简单, 请参考 MLIR 中的相关代码实现。